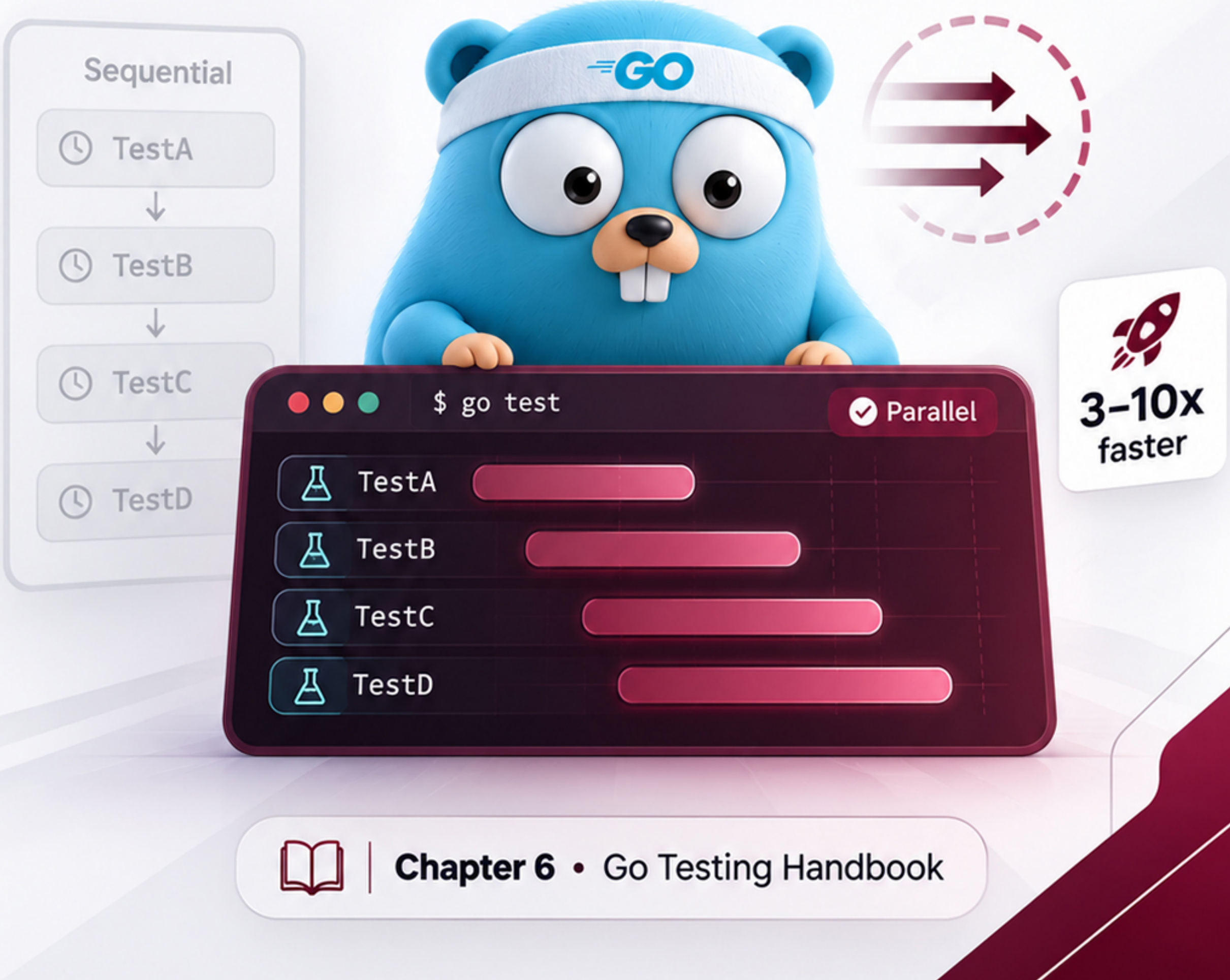
@go_evangelist

Parallel Tests

(t.Parallel)

Run independent tests concurrently.

Faster tests. Same confidence.



Chapter 6 • Go Testing Handbook

Go Testing Handbook | Chapter 6

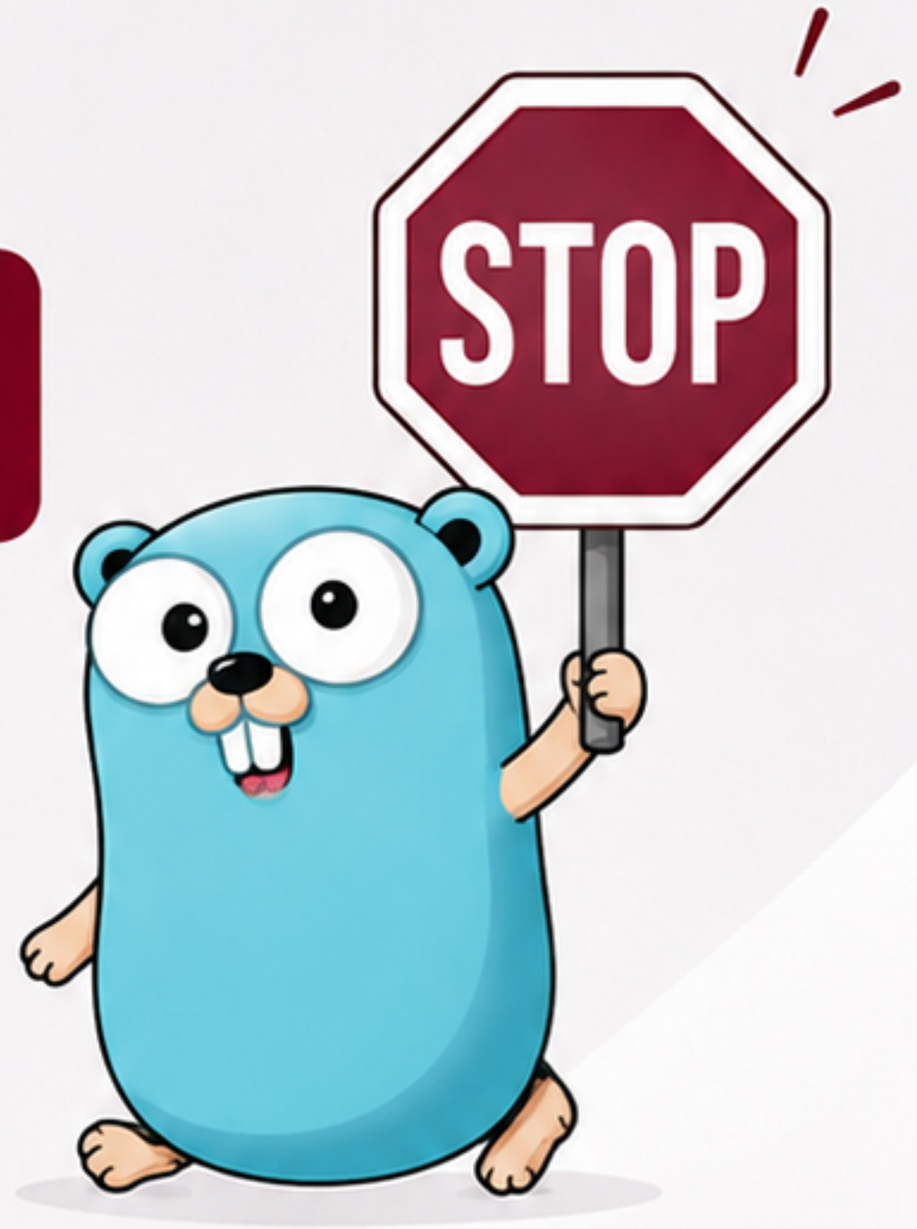
@go_evangelist

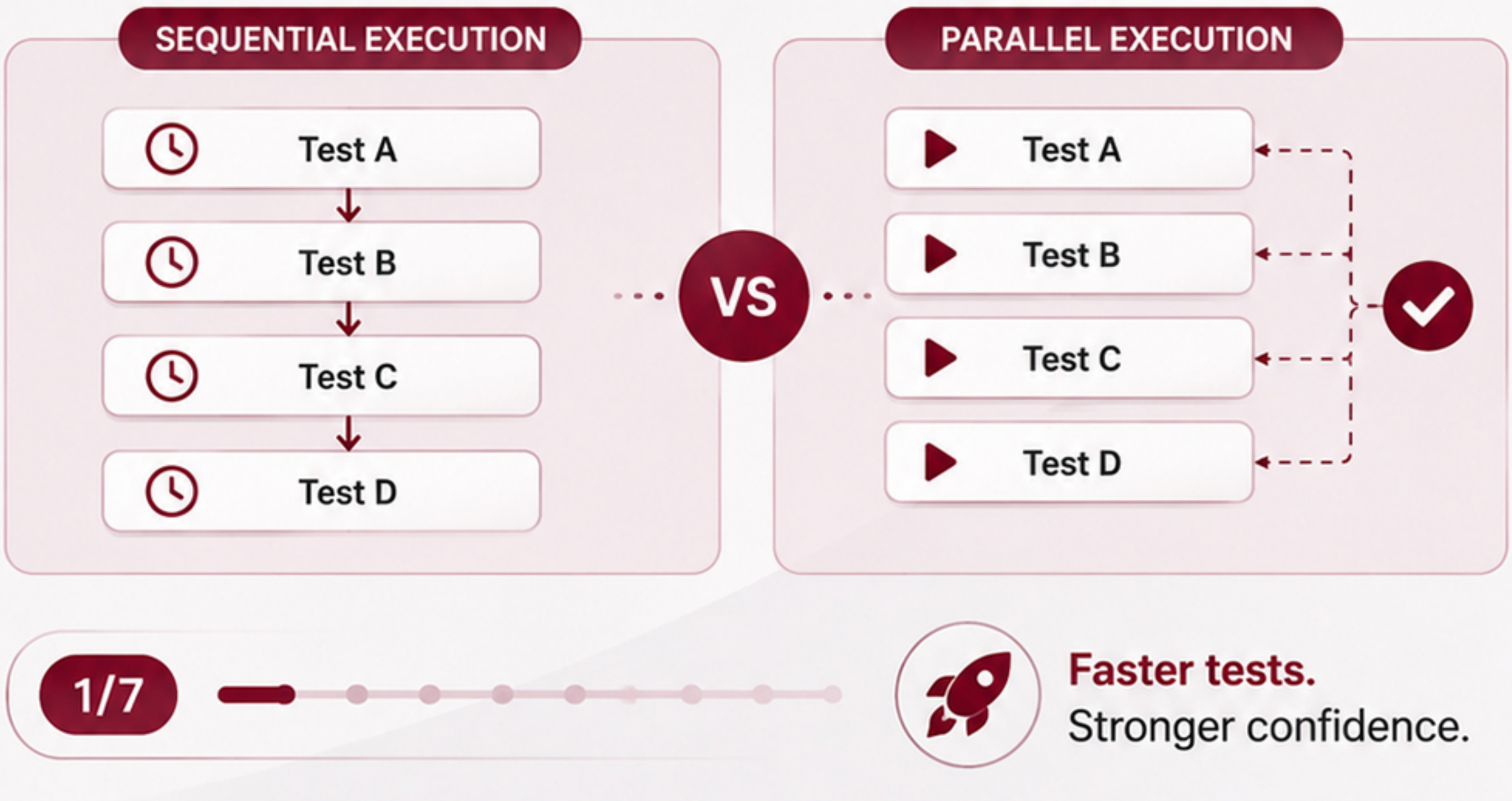
Stop Running Every Test One by One

Sequential tests
wait for each other.

Parallel tests
can run concurrently.

Less waiting. Faster feedback.





1/7

Faster tests.
Stronger confidence.

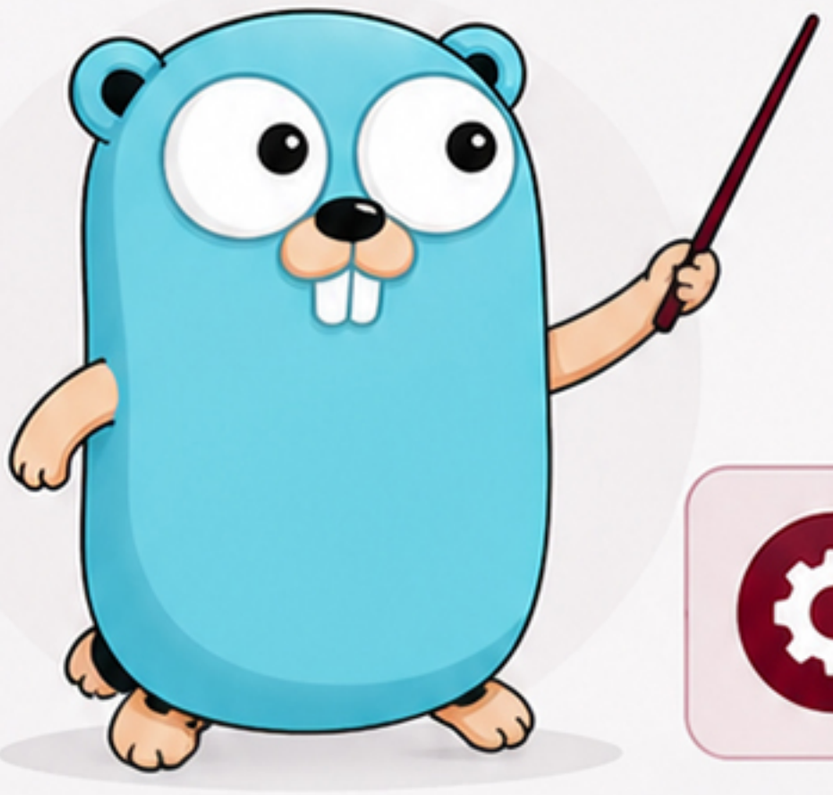
Meet t.Parallel()

Mark **independent tests** as eligible for parallel execution.

add_test.go

```
1  func TestAdd(t *testing.T) {
2      t.Parallel()
3
4      // test logic
5
6  }
7
```

This test may run concurrently with other parallel tests.



Go handles the scheduling.
You focus on writing great tests.

Faster tests.
Stronger confidence.

Parallelize the Right Tests

Best candidates are:

- Independent
- Deterministic
- Read-only
- CPU-bound
- Free from shared state

Independence is the key.

Faster tests.
Stronger confidence.

Shared State = Trouble

Parallel tests can **interfere** through:

- Global variables
- Shared files
- Databases
- Environment variables
- Network ports
- Shared caches

If tests affect each other,
don’t parallelize blindly.

Faster tests.
Stronger confidence.

Parallel Execution Can Reveal Hidden Races

Different tests accessing the same **mutable state** can produce unpredictable results.

Test A
READ

Test B
WRITE

Test A
WRITE

Unexpected Result

Detect them with:

```
> go test -race
```

Faster tests.
Stronger confidence.

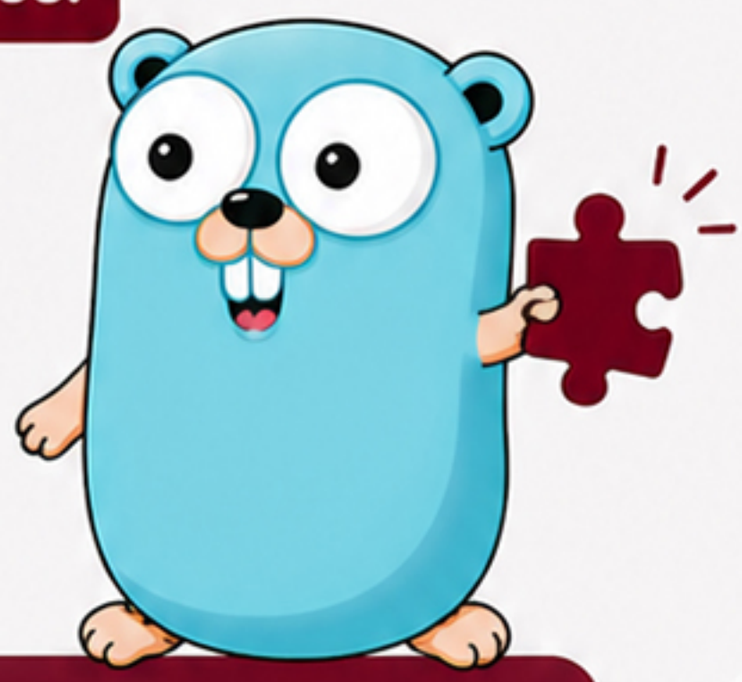
Go Testing Handbook | Chapter 6

@go_evangelist

Table-Driven + Subtests + Parallel

A powerful combination for independent test cases.

Table-Driven Tests + Subtests (t.Run) + Parallel (t.Parallel)



```go
for _, tc := range tests {
    tc := tc

    t.Run(tc.name, func(t *testing.T) {
        t.Parallel()

        // test logic
    })
}
```

Each subtest runs in parallel when independent.

Scalable.
Handle more tests with ease.

Organized.
Clear structure with subtests.

Fast.
Run independent tests concurrently.

Faster tests.
Stronger confidence.

@go_evangelist

Parallel Tests Done Right

- Parallelize only independent tests
- Avoid shared mutable state
- Isolate test resources
- Use `go test -race`
- Keep tests deterministic
- Investigate flaky failures

Speed is valuable.
Reliability comes first.

Read the full chapter on GitHub
Link in bio

END

Next:
Fuzz Testing